Definitional Equalities for the Substitution Calculus using Rewrite Rules

Anonymous Author(s)

Abstract

Choosing a representation of variable binding is a central problem in mechanizing programming language metatheory. De Bruijn indices are canonical, but developments built on them accumulate lemmas about how substitutions compose and how they pass under binders, and every one of those lemmas has to be applied by hand or by a tactic.

The σ-calculus makes parallel substitutions first-class and normalizes them by rewriting. Modeling substitutions as functions turns its rules into theorems, which is what libraries such as Autosubst exploit, but the resulting equalities stay propositional.

We register the σ-calculus as rewrite rules in Agda, so that substitutions are normalized during conversion rather than by a tactic. Two requirements pull against each other. The rules are provable only if the primitives compute, and eligible for rewriting only if they are stuck. Opaque definitions satisfy both. We identify which variant of the calculus can be oriented at all, since native inductive variables and surjective pairing cannot coexist in one rewrite system, and derive the rule set that follows. A generator emits it from a syntax description. Agda certifies local confluence and AProVE termination. Solving the POPLmark Challenge and POPLmark Reloaded on top exercises the result. The laws we cannot register are never needed, and a single law resists our orientation.

CCS Concepts: • **Theory of computation** → *Equational logic and rewriting*.

Keywords: Explicit Substitutions, Term Rewrite Systems, Agda

1 Introduction

Effectively handling variable binding remains a central challenge when mechanizing programming language metatheory. Compared to pen-and-paper proofs, binders in proof assistants are a notorious source of uninteresting technicalities and boilerplate, and many representations have been proposed to contain them, among them locally nameless binders, nominal syntax [37, 38], higher-order abstract syntax, and de Bruijn indices. De Bruijn syntax is canonical and easy to define, but a development built on it still pays for a long list of lemmas about substitution composition and about the interaction of substitution with binders.

CPP '27,
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

On paper, substitution usually replaces a single variable. In a proof assistant one instead works with parallel substitutions, which map variables to terms and replace all of them at once. The σ-calculus [1] makes such a map a first-class object with primitive operations for identity, weakening, extension, composition and lifting, together with a rewrite system that normalizes expressions built from them. The de Bruijn algebra, which models substitutions as functions, provides a semantic model in which those rules become provable [32], and hence usable for equational reasoning. Autosubst [32] and its successor Autosubst 2 [36] generate the operations, the proofs of the laws, and a tactic that normalizes substitutions inside a goal. Autosubst 2 additionally supports intrinsically scoped syntax, packages substitutions over several sorts into vectors, and promotes renamings, the substitutions that map variables to variables, to first-class citizens of the equational theory.

The equalities so obtained are propositional. Normalization is a step the user takes, and every goal that mentions a substitution has to be brought into normal form before it can be closed. Recent support for user-defined rewrite rules in Agda [13, 15] and Rocq [24] offers to remove that step by promoting propositional equalities to definitional ones. Consistency is retained as long as the registered rules are supported by proofs and the resulting system is confluent and terminating [15]. A syntactic condition comes with it. Agda applies a rule by higher-order pattern unification [2] against its left-hand side, and therefore demands that this side be stuck on a defined symbol.

Functions or constructors. There are two ways to model a parallel substitution, and the syntactic condition tells them apart. Modeled as functions, the primitives compute, so the laws are provable, but composition unfolds to a lambda abstraction and no left-hand side is ever stuck. Modeled as syntax, with extension a term former, every left-hand side is stuck by construction. This is the route Wadler takes, and it avoids renamings altogether [39]. What it costs is that a constructor has nothing to compute, so the rule $x\,[\,\mathsf{wk}\,] = \mathsf{suc}\,x$ is no longer derivable, a variable acquires two inequivalent representations, and every law that pushes weakening through a term stays propositional. Agda's opaque definitions [18] give both properties at once. An opaque symbol is stuck outside its block and computes inside it, so one definition can carry the proof of a law and also head that law's left-hand side.

Which calculus can be oriented. Not every version of the σ-calculus is a candidate. The original [1] is confluent on

ground terms but not on open ones, and open terms, those with metavariables, are the setting of any mechanized development. The two variants that repair this differ on the η-laws, that is on surjective pairing, and the choice is forced for us. The complete variant σ_{SP} [31, 35] can afford η only because it has no numerals, since the variable n *is* the expression $0\,[\,\mathsf{wk}\,]^n$. A scoped syntax in a proof assistant has native inductive variables instead, and having them forces $x\,[\,\mathsf{wk}\,] = \mathsf{suc}\,x$, which identifies the two spellings of a variable and thereby destroys the left-hand side of η, which mentions its substitution twice and is not left-linear. Native inductive variables and surjective pairing cannot coexist in one rewrite system. We keep the variables and drop η, and so build on $\sigma_{\Uparrow}$ [16, 19], whose detail section 2.3 gives. Adding the two η-laws to our rule set is refuted for confluence by three independent tools (section 4.3), exactly as this argument predicts.

Contributions

- **A rewrite system.** The σ-calculus with first-class renamings, made definitional in a multi-sorted, intrinsically scoped setting. Every rule is proved before it is registered, so logical consistency is preserved. We derive the rule set rather than list it, as a small base closed under three completion operators, each carrying a side condition that says when an image is not needed, so that the absences are predicted as well as the rules.
- **Machine-checked metatheory of the rule set.** Agda certifies local confluence, and AProVE proves termination and confluence of the first-order export. Both hold at two signature sizes, which is evidence that what is proved is a property of the schema rather than of one language.
- **A generator.** A single-file tool that emits the entire infrastructure from a higher-order abstract syntax description.
- **An evaluation at scale.** The POPLmark Challenge, Parts 1A, 1B, 2A, 2B and 3, and POPLmark Reloaded, solved on generated cores. The laws we cannot register are never needed. One law resists our orientation, and we can say exactly why.

Structure

Section 2 reviews Agda's rewrite rules and opaque blocks, our scoped syntax, and the σ-calculus. Section 3 develops the rule set for System F and explains what makes it registrable. Section 4 reports the generator, the POPLmark evaluation, the external proofs, and future work. Section 5 discusses related work and section 6 concludes.

2 Preliminaries

We review the two Agda features the construction rests on, the scoped syntax we use, and the σ-calculus itself. All Agda code shown in this paper is in the supplement.

2.1 Agda and Relevant Language Features

Agda is a dependently typed functional language and proof assistant based on Martin-Löf type theory [26], where proofs are verified by type checking. Proofs are significantly easier when terms are *definitionally equal*, that is, when they reduce to the same term. When they are not, users must resort to manual equational reasoning with propositional equality, which is cumbersome as Agda lacks automated proof-discharge tactics. Maximizing definitional equalities thus directly reduces manual effort. Beyond standard features such as indexed inductive types and dependent pattern matching [14], this work rests on two more advanced ones.

Rewrite Rules

User-defined rewrite rules [13, 15] extend Agda's normalization-by-evaluation approach for definitional equalities by treating propositional equalities as reduction rules. A proof of $\mathsf{f}\,p_1 \ldots p_n \equiv v$ is eligible if f is a postulate, defined function or constructor, if every variable bound in the statement occurs in a pattern position among $p_1, \ldots, p_n$, and if the left-hand side is stuck. For example, the right identity of addition qualifies:

```
+-id_r : ∀ n → n + 0 ≡ n
+-id_r zero    = refl
+-id_r (suc n) = cong suc (+-id_r n)
```

Once it is registered via {-# REWRITE +-id_r #-}, Agda treats $n + 0$ as definitionally equal to n:

```
_ : ∀ {n} → n + 0 ≡ n
_ = refl
```

Beyond the syntactic restriction, the rule set as a whole has to be well behaved. We rely throughout on the following, which is the criterion under which Agda's rewrite rules are known to be sound [15].

> *If every registered rule is inhabited by a proof of the corresponding propositional equality, and the resulting rewrite system is confluent and terminating, then adding it preserves consistency and subject reduction.*

We discharge the three obligations separately. Every rule is proved before it is registered (section 3.2), Agda's local-confluence check joins every critical pair, and termination is established externally (section 4.3), which with local confluence gives confluence by Newman's lemma. Termination is the obligation that has to leave Agda, because Agda never checks it: a non-terminating rule set simply makes the type checker loop.

Opaque Blocks

Opaque definitions [18] let the user treat code as non-unfolding symbols. A definition inside an opaque block is visible but does not reduce outside it, and the unfolding keyword exposes its computational behavior where that is wanted. The feature exists to improve type checking times, since ordinarily one wants everything to unfold. Here it serves the opposite purpose. Declaring the substitution primitives opaque makes them stuck symbols and thereby qualifies them for rewriting, while the proofs of the rules unfold them and use their computational behavior.

2.2 Multi-Sorted and Scoped Syntax

Many programming languages feature several syntactic categories. System F [17, 28] has three: expressions, types, and kinds[1]. We use intrinsically scoped syntax to prevent the construction of ill-scoped terms. Figure 1 (left) shows the standard implementation, where types are indexed by the number of type variables, expressions by the numbers of type and expression variables, and variables are de Bruijn indices of type Fin n.

Scoped syntax rules out scoping errors without committing to full intrinsic typing, which is hard to work with for larger systems. It has one drawback. The standard representation forces one substitution operation per pair of sorts, such as type-in-type, type-in-expression and expression-in-expression. This is the representation Autosubst 2 chooses, and it explains why that system needs vector substitutions, which bundle the substitutions over the different sorts and come with normalization rules of their own.

To avoid this combinatorial explosion we unify the syntax along two orthogonal dimensions, shown in fig. 1 (right). Variables and terms form a single datatype $S \vdash[\, m \,]\, s$, indexed by a Mode m, a target Sort s and a Scope S, a list of sorts. Variables of type $S \ni s$ are represented across all sorts at once, the length of S being the number of variables in scope and the position of s in S the de Bruijn index. Renamings and substitutions defined over this syntax then operate uniformly across sorts, and vector substitutions become unnecessary.

The mode index is more than a device for sharing definitions. A variable and a term behave differently under a map. Renaming a variable yields a variable, which the applied rules can match again, whereas substituting one yields a term, which they cannot. Several rules therefore exist in two versions pointing in opposite directions, and section 3.2 shows where this is forced.

2.3 A Short History of the Sigma Calculus

Explicit-substitution calculi turn substitutions into syntax, so that $t\,[\,\sigma\,]$ is a term and σ itself can be composed, extended and pushed under binders by rewriting. Three of them matter here.

[1] Kinding for System F is trivial and could be omitted.

σ [1] is the original. Lifting is *defined*, $\Uparrow \sigma := \mathrm{zero}\cdot(\sigma \fatsemi \mathrm{wk})$, and the two η-laws are part of the theory. Both versions of Autosubst build on it.

$\sigma_{\Uparrow}$ [16, 19] keeps $\Uparrow$ primitive and drops η, and thereby recovers confluence on open terms. Its 23 rules, tabulated by Hardin et al. [19], fall into the applied rules, where a variable meets a map, the traversal rules, one per constructor, the map algebra, and a family of $\fatsemi$-companions that exists only because associativity is oriented. This is the calculus we build on.

σ_{SP} [35] adds the η-laws back and is thereby complete, deciding every equality between substitution expressions [31].

Primitive lifting and the absence of η stand or fall together. Eliminate lifting and lift-id, $\Uparrow \mathrm{id} = \mathrm{id}$, stops holding by computation, because $\Uparrow$ then unfolds to $\mathrm{zero}\cdot(\mathrm{id} \fatsemi \mathrm{wk})$ and only η-id reconciles the two. As section 1 argued, η is unavailable to us. Its left-hand side $(\mathrm{zero}[\sigma])\cdot(\mathrm{wk} \fatsemi \sigma)$ mentions σ twice, so any rule that rewrites one occurrence destroys the pattern for the other, and $x\,[\,\mathrm{wk}\,] = \mathrm{suc}\,x$ is such a rule. Completeness and orientability are incompatible here, and a paper about rewriting gives up completeness. The price is four laws, the two η-laws once for renamings and once for substitutions, which we prove but do not register. Section 4.2 reports what that costs in practice.

One further ingredient appears in none of these calculi. Defining instantiation for a substitution requires pushing under binders, which requires weakening the terms in the substitution, which is instantiation again, a loop no termination checker accepts. The standard repair is to define renamings first [3]. Autosubst 2 takes the further step of making renamings first-class and extending the equational theory with rules that relate the two worlds [36]. The extended system is no longer complete, and its confluence was left as a conjecture [35]. We do not settle that conjecture, since the rule set we register is not theirs, but we answer the corresponding question for a concrete two-world system in section 4.3.

Figure 2 previews where this lands. It records the process rather than the result, listing six questions, the answer we took, and for each the alternative we built or attempted first and what forced us off it. Only the first was a matter of taste. Which rules each answer buys is section 3.2's subject and is deliberately kept out of the figure.

For the semantics we use the de Bruijn algebra, which models substitutions as functions and thereby makes the rules provable rather than merely postulated [32]. Variables are natural numbers, terms are $t := x \mid \lambda.\, t \mid t_1\, t_2$, and a substitution maps variables to terms. Reading σ as a list $[t_0, t_1, \dots]$ motivates the primitives:

$$(t\cdot\sigma)(\mathrm{zero}) := t \qquad \mathrm{id}(x) := x$$
$$(t\cdot\sigma)(\mathrm{suc}\,x) := \sigma(x) \qquad \mathrm{wk}(x) := \mathrm{suc}\,x$$

```
data Kind : Set where
  ★ : Kind

data Type (n : Nat) : Set where
  `_       : Fin n → Type n
  ∀[α:_]_  : Kind → Type (suc n) → Type n
  _⇒_      : Type n → Type n → Type n

data Expr (n m : Nat) : Set where
  `_   : Fin m → Expr n m
  λx_  : Expr n (suc m) → Expr n m
  Λα_  : Expr (suc n) m → Expr n m
  _·_  : Expr n m → Expr n m → Expr n m
  _•_  : Expr n m → Type n → Expr n m
```

```
data Sort : Set where
  expr type kind : Sort
Scope = List Sort

data Mode : Set where V T : Mode

data _⊢[_]_ : Scope → Mode → Sort → Set

_⊢_ = _⊢[ T ]_
_∋_ = _⊢[ V ]_

data _⊢[_]_ where
  zero  : (s :: S) ∋ s
  suc   : S ∋ s → (s′ :: S) ∋ s
  `_    : S ∋ s → S ⊢ s
  λx_   : (expr :: S) ⊢ expr → S ⊢ expr
  Λα_   : (type :: S) ⊢ expr → S ⊢ expr
  ∀[α:_]_ : S ⊢ kind → (type :: S) ⊢ type → S ⊢ type
  _·_   : S ⊢ expr → S ⊢ expr → S ⊢ expr
  _•_   : S ⊢ expr → S ⊢ type → S ⊢ expr
  _⇒_   : S ⊢ type → S ⊢ type → S ⊢ type
  *     : S ⊢ kind
```

Figure 1. Classically Scoped vs. Multi-Sorted Scoped Syntax

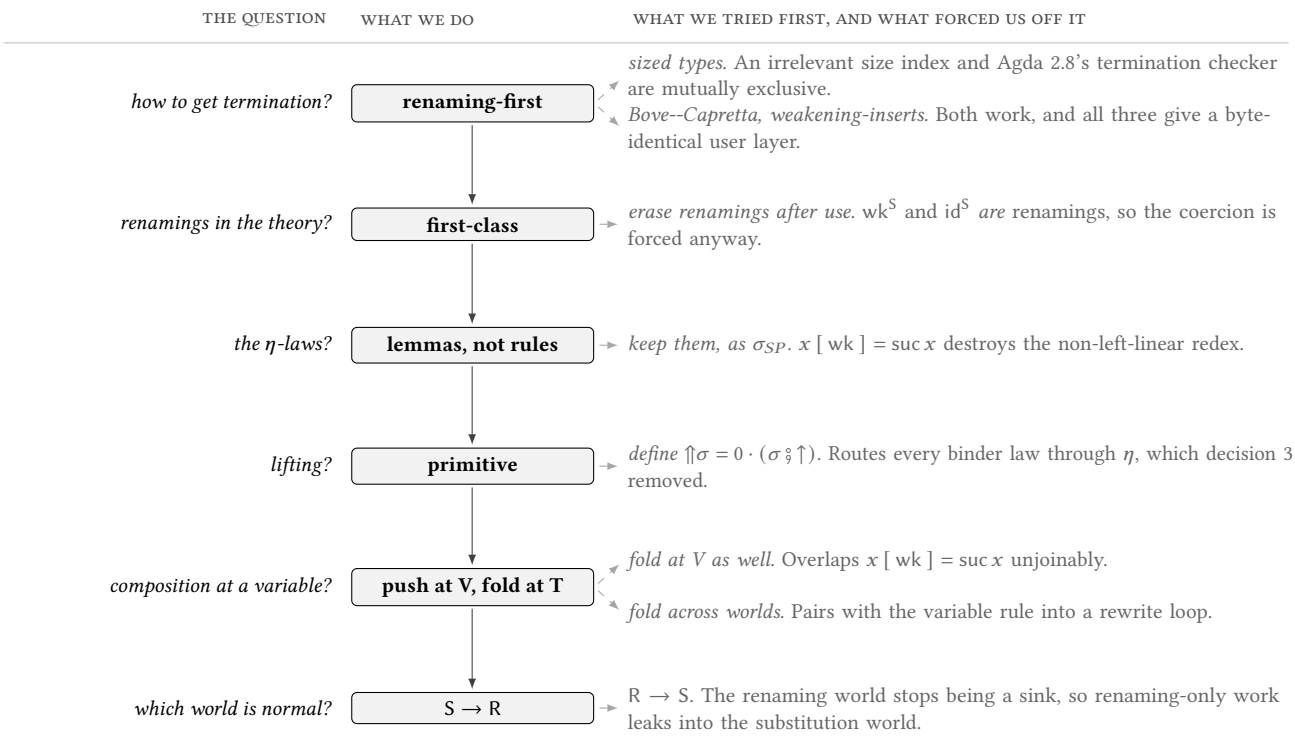

Figure 2. How the rule set was found. Only the first decision is a matter of taste. Every later one is forced, by a critical pair that will not join, a law that can be stated but not oriented, or a redex destroyed before its rule can fire.

Instantiation is then defined mutually with composition and lifting:

$$(\lambda.\, t)\,[\,\sigma\,] = \lambda.(t\,[\,\uparrow\sigma\,]) \qquad (\sigma_1 \fatsemi \sigma_2)(x) = \sigma_1(x)\,[\,\sigma_2\,]$$
$$(t_1\ t_2)\,[\,\sigma\,] = (t_1[\,\sigma\,])\ (t_2[\,\sigma\,]) \qquad \uparrow\sigma := \mathsf{zero}\cdot(\sigma \fatsemi \mathsf{wk})$$

Lifting fixes the bound variable and weakens the rest, and composition looks a variable up in the first map and instantiates the result with the second.

3 Rewriting the Sigma Calculus

Defining instantiation by mutual recursion with composition complicates termination. Agda cannot see that such a definition is structurally recursive, and offers no mechanism to justify it by well-founded recursion, so we follow Adams [3] and define instantiation with renamings first. We then extend the equational theory to handle first-class renamings and to connect the two. We use the multi-sorted, scoped System F of fig. 1 as a running example. The approach applies to any multi-sorted scoped syntax, and to simple intrinsically typed syntax whose type index lives in Set.

3.1 Renaming and Substitution

Figure 3 gives the primitives. Renamings $\_\to^R\_$ map variables of sort s to variables of the same sort, substitutions $\_\to^S\_$ map them to terms, and the operations are the expected ones. We comment only on what the figure cannot show.

Every primitive is opaque, because a rewrite rule needs a stuck left-hand side. This includes both lifting operators. Lifting is *primitive* here rather than defined as $\mathsf{zero}\cdot(\sigma \fatsemi \mathsf{wk})$, which is decision 4 of fig. 2 and the reason the applied and lifting rules do not collapse into η. The identity renaming takes its sort argument explicitly, as Agda frequently fails to infer it. The constants id^S and wk^S are not primitive at all but embedded renamings, obtained through the coercion $\langle\_\rangle$, and are INLINEd so that they do not reduce on a left-hand side.

Two definitions are shaped by later needs. Lifting a substitution is defined by instantiation with the weakening *renaming*, which is what makes the mutual recursion structural. And instantiation $\_[\_]^R$ and $\_[\_]^S$ is defined across both modes, so that looking a variable up in a map is written $x\ [\ \sigma\ ]^S$ rather than as the application $\sigma\ \_\ x$. The rules must be stated using stuck symbols only, with no lambdas and no function application. Section 3.4 returns to this.

3.2 The Sigma Calculus Rules

Figure 4 shows the substitution-world half of the rule set as Agda equalities. The renaming-world duplicates and the $\fatsemi$-companions are in the supplement. Rather than walk the rules one by one, we give the groups. The system is a small base closed under three completion operators, and every one of the 72 registered rules belongs to exactly one group.

The four groups above the line are the content. The three below are the price of design decisions, and each comes with a side condition saying when an image is *not* needed. Those side conditions are what make the count predictive rather than empirical, so we work one of them out.

Orienting associativity right-nests $\fatsemi$, so a rule whose right operand is not a metavariable stops matching once a continuation is appended. Take lift-wk and overlap it with assoc on $((\langle\mathsf{wk}\rangle \fatsemi (\Uparrow \sigma)) \fatsemi \tau$. Reducing the inner redex first gives $(\sigma \fatsemi \langle\mathsf{wk}\rangle) \fatsemi \tau$, which assoc takes to $\sigma \fatsemi (\langle\mathsf{wk}\rangle \fatsemi \tau)$. Reducing with assoc first gives $\langle\mathsf{wk}\rangle \fatsemi ((\Uparrow \sigma) \fatsemi \tau)$, where nothing matches any more. The pair is closed by registering exactly that term as the continuation image of lift-wk, and six other base rules need the same treatment.

#	group	what it is for
16	traversal	push an application through one constructor, the only signature-dependent group, two rules per constructor
9	lookup	evaluate a constructor-headed map at a variable, that is, the definitions of wk, $\cdot$ and $\Uparrow$
16	map algebra	normalize a composed map, once per world
7	fusion	turn two traversals into one, in all four renaming/substitution combinations, the two mixed cases being Autosubst 2's `compRenSubst` and `compSubstRen` [36]
7	continuation	assoc right-nests $\fatsemi$, so a rule matching a non-variable right operand can no longer see it inside a chain
8	mode V	a lookup cannot evaluate an accumulator, so at mode V the map must be driven back to constructor-headed form
9	coercion	$\langle\cdot\rangle$-lift and $\langle\cdot\rangle$-comp erase the $\Uparrow$ and $\fatsemi$ heads that the σ-rules match on, so every such rule needs a coerced image

interaction, $\langle\mathsf{wk}\rangle \fatsemi (t\cdot\sigma) = \sigma$, has the identical overlap and needs no image. Its continued redex is $\langle\mathsf{wk}\rangle \fatsemi ((t\cdot\sigma) \fatsemi \tau)$, and here the right operand does reduce. dist turns it back into cons-shaped form, after which interaction matches again. That is the side condition. An image is needed when the right operand is not a metavariable *and* the continued redex has no inner reduction. Five rules satisfy the first half and fail the second, namely comp-id$_r$ in both worlds, interaction, lift-cons and its coercion image, and the supplement checks each escape as a `refl` assertion. The rule set matches the predicate exactly, absences included.

Three rules deserve to be named individually.

coincidence, $t\ [\ \langle\xi\rangle\ ]^S = t\ [\ \xi\ ]^R$, is the orientation decision. It makes the renaming world the normal form, so that renaming-only work never leaves it, and everything in the coercion group follows from that direction.

compositionality^{RR} exists twice, pointing in opposite directions:

$$(t\ [\ \xi_1\ ])\ [\ \xi_2\ ] = t\ [\ \xi_1 \fatsemi \xi_2\ ] \qquad \text{at mode T, } \textit{fold}$$
$$x\ [\ \xi_1 \fatsemi \xi_2\ ] = (x\ [\ \xi_1\ ])\ [\ \xi_2\ ] \qquad \text{at mode V, } \textit{push}$$

This is the only place where the two modes disagree on a direction, and they must. Renaming preserves the mode, so $x\ [\ \xi\ ]$ is again a variable and again a subject for the applied rules. Folding at a variable would leave a pending composition that no lookup rule can evaluate, and it would overlap $x\ [\ \mathsf{wk}\ ] = \mathsf{suc}\ x$ unjoinably. At a term, accumulating is instead what keeps the traversal linear. In the substitution

```
_→R_ : Scope → Scope → Set
S₁ →R S₂ = S₁ →[ V ] S₂
opaque
  idR : S →R S
  idR _ x = x
  wkR : ∀ s′ → S →R (s′ :: S)
  wkR _ _ x = suc x
  _·R_ : S₂ ∋ s → S₁ →R S₂ → (s :: S₁) →R S₂
  (x ·R ξ) _ zero  = x
  (_ ·R ξ) _ (suc x) = ξ _ x
  _↑R_ : S₁ →R S₂ → ∀ s → (s :: S₁) →R (s :: S₂)
  (ξ ↑R _) _ zero = zero
  (ξ ↑R _) _ (suc x) = suc (ξ _ x)
  _[_]R : S₁ ⊢[ m ] s → S₁ →R S₂ → S₂ ⊢[ m ] s
  _[_]R {m = V} x ξ = ξ _ x
_⨟R_ : S₁ →R S₂ → S₂ →R S₃ → S₁ →R S₃
(ξ₁ ⨟R ξ₂) _ x = (ξ₁ _ x) [ ξ₂ ]R
```

```
_→S_ : Scope → Scope → Set
S₁ →S S₂ = S₁ →[ T ] S₂
opaque
  ⟨_⟩ : S₁ →R S₂ → S₁ →S S₂
  ⟨ ξ ⟩ _ x = ‘ (x [ ξ ]R)
idS : S →S S
idS = ⟨ idR ⟩
wkS : ∀ s′ → S →S (s′ :: S)
wkS s′ = ⟨ wkR s′ ⟩
opaque
  _·S_ : S₂ ⊢ s → S₁ →S S₂ → (s :: S₁) →S S₂
  (t ·S σ) _ zero  = t
  (t ·S σ) _ (suc x) = σ _ x
opaque
  unfolding _·S_
  _[_]S : S₁ ⊢[ m ] s → S₁ →S S₂ → S₂ ⊢ s
  _↑S_ : S₁ →S S₂ → ∀ s → (s :: S₁) →S (s :: S₂)
  (σ ↑S _) _ zero = ‘ zero
  (σ ↑S _) _ (suc x) = (σ _ x) [ wkR _ ]R
  _[_]S {m = V} x σ = σ _ x
_⨟S_ : S₁ →S S₂ → S₂ →S S₃ → S₁ →S S₃
(σ₁ ⨟S σ₂) _ x = (σ₁ _ x) [ σ₂ ]S
```

Figure 3. The Primitives for Renaming and Substitution

```
coincidence-var : x [ ⟨ ξ ⟩ ]S ≡ ‘ (x [ ξ ]R)
def-·S-zero  : zero [ (t ·S σ) ]S ≡ t
def-·S-suc   : (suc {s′ = s′} x) [ (t ·S σ) ]S ≡ x [ σ ]S
def-↑S-zero : zero [ (σ ↑S s) ]S ≡ ‘ zero
def-↑S-suc   : (suc x) [ (σ ↑S s) ]S ≡ x [ (σ ⨟ ⟨ wkR s ⟩) ]S
interact     : ⟨ wkR s ⟩ ⨟ (t ·S σ) ≡ σ
comp-idₗ     : ⟨ idR {S₁} ⟩ ⨟ σ ≡ σ
comp-idᵣ     : σ ⨟ ⟨ idR ⟩ ≡ σ
lift-wk      : ⟨ wkR s ⟩ ⨟ (σ ↑S s) ≡ σ ⨟ ⟨ wkR s ⟩
assoc        : (σ₁ ⨟ σ₂) ⨟ σ₃ ≡ σ₁ ⨟ (σ₂ ⨟ σ₃)
dist         : (t ·S σ₁) ⨟ σ₂ ≡ (t [ σ₂ ]S) ·S (σ₁ ⨟ σ₂)
lift-cons    : (σ ↑S s) ⨟ (t ·S τ) ≡ t ·S (σ ⨟ τ)
lift-dist-compSS : ((σ₁ ↑S s) ⨟ (σ₂ ↑S s)) ≡ ((σ₁ ⨟ σ₂) ↑S s)
```

```
coincidence : ∀ (t : S ⊢ s) (ξ : S →R S₂) → t [ ⟨ ξ ⟩ ]S ≡ t [ ξ ]R
⟨⟩-comp : ⟨ ξ₁ ⟩ ⨟ ⟨ ξ₂ ⟩ ≡ ⟨ ξ₁ ⨟R ξ₂ ⟩
⟨⟩-split-⨟ : ⟨ ξ₁ ⨟R ξ₂ ⟩ ⨟ σ ≡ ⟨ ξ₁ ⟩ ⨟ (⟨ ξ₂ ⟩ ⨟ σ)
⟨⟩-lift      : (⟨ ξ ⟩ ↑S s) ≡ ⟨ ξ ↑R s ⟩
inst-x : (‘ x)        [ σ ]S ≡ x [ σ ]S
inst-λ : (λx e)       [ σ ]S ≡ λx (e [ (σ ↑S _) ]S)
inst-Λ : (Λα e)       [ σ ]S ≡ Λα (e [ (σ ↑S _) ]S)
inst-∀ : (∀[α: k ] t) [ σ ]S ≡ ∀[α: k [ σ ]S ] (t [ (σ ↑S _) ]S)
inst-· : (e₁ · e₂)    [ σ ]S ≡ (e₁ [ σ ]S) · (e₂ [ σ ]S)
inst-• : (e • t)      [ σ ]S ≡ (e [ σ ]S) • (t [ σ ]S)
inst-⇒ : (t₁ ⇒ t₂) [ σ ]S ≡ (t₁ [ σ ]S) ⇒ (t₂ [ σ ]S)
inst-* : * {S = S}    [ σ ]S ≡ *
compositionalitySS : ∀ (x/t : S₁ ⊢[ m ] s)
  {σ₁ : S₁ →S S₂} {σ₂ : S₂ →S S₃} →
  (x/t [ σ₁ ]S) [ σ₂ ]S ≡ x/t [ (σ₁ ⨟ σ₂) ]S
```

Figure 4. The Rewrite System for the Sigma Calculus with First-Class Renamings

world the question does not arise, because a substituted variable is a term and no applied rule can match it.

⟨·⟩-comp is the one completion image in the system that cannot be taken. The closure demands it, but the general image is the exact inverse of ⟨·⟩-split, so registering both loops. What we register instead is that image restricted to the prefixes on which the renaming world can still make progress. There are three of them, interaction^{R}, lift-wk^{R} and lift-dist-comp^{RR}, and they are exactly the three renaming-world rules that need continuation images of their own. The same set answers both questions.

Nine further laws are proved but not registered. Four are the η-laws, for the left-linearity reason of section 2.3. Two are dist^{R} and lift-cons^{R}, whose variable instances collide with

the lookup rules under *push at V*. Two are the definitions of lifting, which we keep primitive. The last, lift-id, is not excluded but subsumed, since its left-hand side is a strict instance of $\langle\cdot\rangle$-lift's, which already sends it to the same normal form.

All rules are proved before being registered, which is what preserves consistency. The proofs unfold the opaque definitions and use function extensionality, which is conservative over Agda's theory [20]. We refer the reader to the supplement.

3.3 What the Rules Buy

We exercise the rule set on the standard metatheory of the System F of fig. 1, against a typing judgment $\_\vdash\_:\_$ and a call-by-value small-step semantics $\_\hookrightarrow\_$. A context Ctx S assigns to every variable in scope S a type one sort up, so $\Gamma \vdash e : t$ reads as usual. Figure 5 collects everything the argument below depends on.

Substitution enters this metatheory in exactly three places, shown top left. $\vdash\lambda$ stores the result type of a function *weakened*, since the body is typed in a longer context. $\vdash\bullet$ concludes at a single substitution $t'\,[\,t\,]_0$, because instantiating a polymorphic type substitutes for its bound variable. And β-λ contracts a redex to $e_1\,[\,e_2\,]_0$. Each of the three puts a substitution somewhere a proof will later have to move it, and each therefore creates a demand for one law.

Top right are the two statements. A substitution is well typed, written $\sigma : \Gamma_1 \to^S \Gamma_2$, when it sends every variable x typed t in Γ_1 to a term $x\,[\,\sigma\,]^S$ typed $t\,[\,\sigma\,]^S$ in Γ_2. The type is substituted along with the term, which is what makes the notion compose. sub-pres then says that applying such a substitution to a well typed term preserves typing, and sr is subject reduction. Both are the expected inductions and use two helpers, shown middle right. $\vdash\uparrow^S$ lifts a well typed substitution under a binder, and is proved from the corresponding statement for renamings, which is what keeps the recursion structural. $\vdash[]$ turns a well typed term into the well typed single substitution that the two β-cases need.

Middle left are the three laws the three obligations create a demand for. A de Bruijn development states and applies each of them by hand. Here each holds by refl, because each is a normalization rather than a lemma: the rules turn both sides into the same normal form, and refl sees that they agree. Written out for subst-commute, the classic substitution lemma, the two branches are

$$
\begin{array}{ll}
(t\,[\,\Uparrow\sigma\,])\,[\,t'[\sigma]\,]_0 \;=\; (t\,[\,\Uparrow\sigma\,])\,[\,t'[\sigma]{\cdot}\mathsf{id}\,] & \\
\to\; t\,[\,(\Uparrow\sigma)\fatsemi(t'[\sigma]{\cdot}\mathsf{id})\,] & \text{compositionality} \\
\to\; t\,[\,t'[\sigma]{\cdot}(\sigma\fatsemi\mathsf{id})\,] & \text{lift-cons} \\
\to\; t\,[\,t'[\sigma]{\cdot}\sigma\,] & \text{comp-id}_r \\
(t\,[\,t'\,]_0)\,[\,\sigma\,] \;=\; (t\,[\,t'{\cdot}\mathsf{id}\,])\,[\,\sigma\,] & \\
\to\; t\,[\,(t'{\cdot}\mathsf{id})\fatsemi\sigma\,] & \text{compositionality} \\
\to\; t\,[\,t'[\sigma]{\cdot}(\mathsf{id}\fatsemi\sigma)\,] & \text{dist} \\
\to\; t\,[\,t'[\sigma]{\cdot}\sigma\,] & \text{comp-id}_l
\end{array}
$$

Five rules and no user intervention. Every intermediate term above is checked against the endpoint in the supplement, so the trace is the one Agda takes. A normal form is reached when no traversal can be pushed further, no composition can be fused, and no embedded renaming is left in the substitution world. On a closed term the substitution disappears entirely. On an abstract t it survives as a single application to a right-nested, identity-free map, as here.

The bottom of the figure closes the loop, and each clause can now be read against the rule above it. In the $\vdash\lambda$ case the induction hypothesis types the body at $(\text{weaken}\ t')\,[\,\Uparrow\sigma\,]$, since that is where $\vdash\lambda$ put it, while $\vdash\lambda$ demands weaken $(t'\,[\,\sigma\,])$ to rebuild the abstraction. That is wk-comm. In the $\vdash\bullet$ case the goal is $(t'\,[\,t\,]_0)\,[\,\sigma\,]$ and the three sub-derivations supply $(t'\,[\,\Uparrow\ \sigma\,])\,[\,t\,[\,\sigma\,]\,]_0$, which is subst-commute. In the β-case for $\lambda\mathsf{x}\_$, $\vdash\lambda$ has stored the result type weakened, so sub-pres delivers the contractum at $(\text{weaken}\ t_2)\,[\,e_2\,]_0$ where the goal is plain t_2, which is wk-cancel.

Every other clause of either proof needs nothing, and for two different reasons. $\vdash\Lambda$, $\vdash\cdot$ and $\vdash^*$ conclude at a type the substitution never has to be moved through, so no equation arises. The β-case for $\Lambda\alpha\_$ does face one, $t'\,[\,t\,]_0$ against $t'\,[\,t{\cdot}\mathsf{id}\,]$, but the two are the same term by the definition of single substitution. Since the three that do arise are conversions as well, every clause of both proofs is a bare constructor application.

The three are not special. The same holds for the laws we do not need here, among them that two single substitutions can be exchanged and that lifting distributes over composition, and for their renaming-world twins and the two mixed laws relating the worlds, which a development without first-class renamings cannot even state. The supplement checks all of them, and all of them are refl.

One case does need help, and the cause is the rewriting itself. In the two β-cases we pass σ explicitly, writing sub-pres $\{\sigma = e_2 \cdot^S \mathsf{id}^S\}$. By the time Agda sees the goal, the rules have normalized the type index, so σ can no longer be recovered by unifying against $e\,[\,\sigma\,]^S$. Definitional equality and inferability pull against each other here. The more of the index is normalized before unification, the less of it is left to read a metavariable off. The cost is one implicit argument, and no substitution law is applied by hand.

```
⊢λ :
  (t ::ₜ Γ) ⊢ e : (weaken t′) →
  Γ ⊢ (λx e) : (t ⇒ t′)
⊢• :
  Γ ⊢ e : (∀[α: k ] t′) →
  Γ ⊢ t : k →
  (k ::ₜ Γ) ⊢ t′ : k′ →
  Γ ⊢ (e • t) : (t′ [ t ]₀)
β-λ :
  Val e₂ →
  ((λx e₁) · e₂) ↪ (e₁ [ e₂ ]₀)

wk-cancel : ∀ {t : S ⊢ s} {t′ : S ⊢ s′} → (weaken t) [ t′ ]₀ ≡ t
wk-cancel = refl
wk-comm : ∀ {t : S₁ ⊢ s} {σ : S₁ →ˢ S₂} →
  (weaken {s′ = s′} t) [ (σ ↑ˢ s′) ]ˢ ≡ weaken (t [ σ ]ˢ)
wk-comm = refl
subst-commute : ∀ {t : (s′ :: S₁) ⊢ s} {t′ : S₁ ⊢ s′}
  {σ : S₁ →ˢ S₂} →
  (t [ (σ ↑ˢ s′) ]ˢ) [ t′ [ σ ]ˢ ]₀ ≡ (t [ t′ ]₀) [ σ ]ˢ
subst-commute = refl

_:_→ˢ_ : S₁ →ˢ S₂ → Ctx S₁ → Ctx S₂ → Set
_:_→ˢ_ {S₁} σ Γ₁ Γ₂ = ∀ s (x : S₁ ∋ s) (t : S₁ :⊢ s) →
  Γ₁ ∋ x : t → Γ₂ ⊢ (x [ σ ]ˢ) : (t [ σ ]ˢ)
sub-pres : ∀ {σ : S₁ →ˢ S₂} {Γ₁ : Ctx S₁} {Γ₂ : Ctx S₂}
  {e : S₁ ⊢ s} {t : S₁ :⊢ s} →
  Γ₁ ⊢ e : t → σ : Γ₁ →ˢ Γ₂ →
  Γ₂ ⊢ (e [ σ ]ˢ) : (t [ σ ]ˢ)
sr : Γ ⊢ e : t → e ↪ e′ → Γ ⊢ e′ : t

⊢↑ˢ : ∀ {σ : S₁ →ˢ S₂} {Γ₁ : Ctx S₁} {Γ₂ : Ctx S₂} →
  σ : Γ₁ →ˢ Γ₂ → (t : S₁ :⊢ s) →
  (σ ↑ˢ s) : (t ::ₜ Γ₁) →ˢ ((t [ σ ]ˢ) ::ₜ Γ₂)
⊢[] : ∀ {Γ : Ctx S} {e : S ⊢ s} {t : S :⊢ s} →
  Γ ⊢ e : t → (e ·ˢ idˢ) : (t ::ₜ Γ) →ˢ Γ

-- the induction hypothesis types the body at (weaken t′) [ σ ↑ˢ _ ]ˢ,
-- while ⊢λ demands weaken (t′ [ σ ]ˢ).  Discharged by wk-comm.
sub-pres {σ = σ} (⊢λ ⊢e) ⊢σ = ⊢λ (sub-pres {σ = σ ↑ˢ _} ⊢e (⊢↑ˢ {σ = σ} ⊢σ _))
-- ⊢• concludes at t′ [ t ]₀, so the two sides are
-- (t′ [ σ ↑ˢ _ ]ˢ) [ t [ σ ]ˢ ]₀  and  (t′ [ t ]₀) [ σ ]ˢ.
-- Discharged by subst-commute.
sub-pres {σ = σ} (⊢• ⊢e ⊢t ⊢t′) ⊢σ = ⊢• (sub-pres {σ = σ} ⊢e ⊢σ) (sub-pres {σ = σ} ⊢t ⊢σ)
                                      (sub-pres {σ = σ ↑ˢ _} ⊢t′ (⊢↑ˢ {σ = σ} ⊢σ _))
-- ⊢λ stores the result type weakened, so the redex is typed at
-- (weaken t₂) [ e₂ ]₀ where the goal is t₂.  Discharged by wk-cancel.
sr (⊢· {e₂ = e₂} (⊢λ ⊢e₁) ⊢e₂) (β-λ v₂) =
  sub-pres {σ = e₂ ·ˢ idˢ} ⊢e₁ (⊢[] ⊢e₂)
```

Figure 5. Everything the three interesting clauses depend on. Top left, the two typing rules and the reduction rule whose types carry a substitution. Top right, the two statements and the notion of a well typed substitution. Middle left, the three laws this creates a demand for, each holding by refl; middle right, the two helpers the clauses call. Bottom, the only three clauses of either proof in which a law is spent, with the law named. Every other clause needs none.

3.4 Why the Definitions Must Not Unfold

Blocking the reduction of the primitives is what lets us register every rule. We now explain why, and why the alternatives fail.

The immediate problem is that composition unfolds to a lambda abstraction. Unfolded, the left-hand side of comp-id$_l$ reduces to $\langle\ \mathrm{id}^R\ \rangle$, which is not a stuck symbol. But once composition is blocked, we still have to say how a variable is looked up in a composite.

Our first attempt kept function application for lookup, by rewriting $(\sigma_1 \fatsemi \sigma_2)\ \_\ x \equiv (x\ [\ \sigma_1\ ]^S)\ [\ \sigma_2\ ]^S$. Agda accepts this alongside the interaction rules, but then misbehaves during higher-order pattern unification and stops applying the η-laws, which propositional reasoning still discharges by hand. We think Agda should reject the construction rather than accept it and degrade.

Instead we use the instantiation operator for lookup, which has to be propagated through every rule. Function application and lambda-built maps remain available to the user but yield fewer definitional equalities, so we recommend instantiation for lookup and the two composition operators for everything else.

Using Vectors instead of Functions

A vector-like data type whose extension is a constructor is the other candidate representation. As in section 1 it is stuck by construction, so it needs no opacity machinery, and composition by case distinction on the vector is a legal rewrite pattern.

The difference is not that η has to go, since we drop η too, for the independent reason of section 2.3. The difference is what can be proved. In our development all 72 registered rules and the nine unregistered ones are theorems, discharged against the function model, and that is what makes the exclusions measurable rather than merely asserted. Because we can state and prove the η-laws, we can also report that they are never once applied by hand across the whole POPLmark development. Under a constructor encoding those laws are not derivable, since a constructor has no computation rules to derive them from, so they would have to be postulated or the type quotiented. Postulating forfeits exactly the consistency argument that makes us prove rules before registering them, and quotienting reintroduces the transports we set out to remove.

The vector representation remains the right choice where functions are not available, notably when formalizing type theory in type theory [7], and it is what the uniform renaming/substitution treatments [6, 27] of section 5 use.

The original σ-calculus treats its symbols purely syntactically, describing lookup itself through instantiation. The de Bruijn algebra then supplied a semantic model in functions, which made the rules provable and usable for automated propositional reasoning. This work does both at once. The model proves the laws, and the syntactic reading makes them apply.

4 Implementation and Evaluation

4.1 Code Generation

Only the traversal group depends on the signature. Everything else is schematic in the constructor list. The construction is therefore mechanizable, and we provide a script that translates a higher-order abstract syntax (HOAS) description into Agda code including the rules ready for rewriting. Figure 6 shows the description for our System F example. This mirrors what Autosubst 2 does for Rocq, where the generated output is the scoped syntax together with the substitution lemmas that need induction over it.

Several of Autosubst 2's features come for free. Vector substitutions are unnecessary and mutually recursive definitions work unchanged, both because the syntax is multi-sorted. Inline Agda code, such as lists or products of terms, can be encoded with additional sorts at the cost of standard library support. Two features are missing, a distinction between sorts with and without binders, which an additional index on Sort would provide, and variadic syntax, which could be encoded through arguments to the sort constructors [29].

Adding a constructor to a language is mechanical, since only the traversal group depends on it and it contributes exactly two rules there. Binders that bind several variables at once are not free in the same way. They are handled by a lifting operation that lifts by a *list* of sorts, and the completion has to be redone at each lifting level, which is what the iterated $\Uparrow^*$ family costs. This is why the generated cores carry more rules than the 72 of section 3.2: 77 for STLC, 83 with sums, 87 for $F_{<:}$, 101 with records and 111 with pattern matching. Everything above 72 is the $\Uparrow^*$ family and the extra traversal rules of the larger signatures.

The script is a single Python file with strict type checking enabled, comprising a parser for HOAS descriptions and ad-hoc code generation, and it carries all boilerplate Agda code so that others can try it standalone. It is a proof of concept rather than a library. Beside the languages of section 4.2 we generate a dozen files from the HOAS descriptions in the Autosubst 2 repository.

4.2 POPLmark and POPLmark Reloaded

To test the construction where substitution reasoning actually hurts, we solved the POPLmark Challenge, Parts 1A, 1B, 2A, 2B and 3, and POPLmark Reloaded including its sums extension, on top of generated cores. The development is roughly 9,100 lines of Agda, 6,400 of them excluding blanks and comments, of which 2,800 are generated infrastructure and 3,500 hand-written metatheory. It spans five languages, $F_{<:}$, $F_{<:}$ with records, $F_{<:}$ with pattern matching, STLC and STLC with sums, and `fun-ext` is its only postulate.

The point of comparison is the substitution reasoning a de Bruijn development normally pays for. In the Rocq solutions by the Autosubst authors the σ-laws are applied by an `asimpl` tactic, invoked explicitly wherever substitutions must be normalized. Here there is no such step, because normalization happens during conversion. Section 5 places the coverage among published solutions. Three measurements support the claim, and all three are properties of the whole development rather than of a chosen example.

- The η-laws, the ones we cannot register, are applied by hand zero times. Their absence is not felt.
- No proof uses a `rewrite` clause. Thirty transports remain across the metatheory modules, of which exactly two are along a substitution equation, and both are the same lemma.
- That lemma, which we call `[]-as-ren`, is a single law, and its failure is not an accident of the tooling. It says

$$t\,[\,\text{`}x\,]_0 \;=\; t\,[\,x{\cdot}\mathsf{id}\,]^{\mathrm{R}}$$

  and the two sides are two *distinct* normal forms. The left one unfolds to $t\,[\,(\text{`}x){\cdot}\mathsf{id}\,]$, which is cons-shaped, while coincidence needs a syntactic $\langle\xi\rangle$ to collapse into the renaming world. The rule that would bridge

```
kind : Type
type : Type
expr : Type

-- the constructors for kind
star : kind

-- the constructors for type
arr : type -> type -> type
all : kind -> (type -> type) -> type

-- the constructors for expr
app  : expr -> expr -> expr
lam  : type -> (expr -> expr) -> expr
tapp : expr -> type -> expr
tlam : kind -> (type -> expr) -> expr
```

Figure 6. Higher-order abstract syntax specification for System F

them is $(\text{`}\, x)\cdot\langle\xi\rangle \longrightarrow \langle x\cdot\xi\rangle$, in the same S $\longrightarrow$ R direction as the rest of the coercion group. Registering it costs four non-joinable pairs, and the one that survives every attempt to complete the system is

$$(x\,[\Uparrow\xi\,])\,[\,(\text{`}\, y)\cdot\langle\xi_1\rangle\,]$$

whose reducts meet only if lift-cons$^{\mathsf{R}}$ can fire, which needs composition at a variable to fold. It pushes, because folding there overlaps $x\,[\,\mathsf{wk}\,] = \mathsf{suc}\,x$ unjoinably. The one hand-proved substitution fact is therefore the push-at-V decision seen from the substitution side, not an independent gap, and we supply the missing join as an induction on the term.

Two costs the system does not remove are worth reporting. First, a function defined by recursion over the syntax does not commute with substitution for free. The size and shape measures the Challenge needs still require their own induction, since with an abstract argument the traversal rules do not fire. Second, living under a rewrite system constrains what may be matched, not only what may be proved. An earlier version of the transitivity proof for subtyping recursed on $Q\,[\,\xi\,]^{\mathsf{R}}$ to make the induction structural in Q. It type checks in the ordinary sense, but is rejected by `--local-confluence-check`, because matching on Q puts $(\forall\ldots)\,[\,\xi\,]^{\mathsf{R}}$ into the clause's left-hand side, where it overlaps a traversal rule unjoinably.

4.3 External Verification

Agda's `--local-confluence-check` certifies that every critical pair joins. It says nothing about termination, which Agda does not check for rewrite rules at all. We therefore export the system as a first-order term rewriting system, with scope indices erased, and hand the resulting 75 rules to external provers. AProVE proves termination in 23.9 s and confluence in 24.9 s. Local confluence plus termination gives confluence by Newman's lemma, so the two results agree by independent routes, and the obligations of section 2.1 are discharged.

The system does *not* pass Agda's global `--confluence-check`, and it is worth saying why, because an artifact reviewer will flip the pragma. Turning it on reports 277 errors, every one of them `RewriteAmbiguousRules` and none of them a missing rule or a non-joinable pair. The global check asks that no two left-hand sides overlap at all, which is a stricter condition than confluence, and a σ-calculus cannot satisfy it. assoc overlaps every rule whose left-hand side mentions $\fatsemi$, by construction. Overlaps are exactly what the local check examines, as critical pairs, and it finds every one of them joinable. This is a property of the calculus, not of our encoding or of the generator, and the hand-written core fails the global check too.

Cost. Definitional equalities are paid for at type-checking time. Measured on one core under Agda 2.8, a module that does nothing but import a generated core costs about 70 s, against 1 s for a comparable module importing only the standard library, and the figure is flat in the size of the rule set, 69 s at 77 rules and 66 s at 111. Against per-module times of 74 to 120 s across the development, that fixed import cost dominates and the proof content is cheap. The exception is the Part 3 animation, where the rules are doing real evaluation rather than conversion, and which takes about 76 minutes.

Two controls make the verdicts meaningful. Both properties hold at two signature sizes, three and eight constructors, which is what licenses the generator claim. And the setup has a falsifier that fires, since adding the two η-laws is refuted for confluence by three independent tools, as section 2.3 predicts. Evidence at two sizes is not a proof schematic in the signature, and obtaining one would mean decomposing the system along the lines of compositional confluence criteria [33].

4.4 Future Work

Opaque Definitions for Rewriting. The construction suggests a general recipe for integrating normalization for suitable algebraic structures into a proof assistant: define symbols for the structure; establish a confluent rewrite system over them; instantiate the structure with an underlying model and prove the laws inside opaque blocks; and, with the model hidden, REWRITE the equations.

Classical sets are one candidate. Reasoning about them in Agda usually means lists up to permutation together with a uniqueness predicate, which is tedious and generates a great deal of uninteresting code. Their characteristic function supplies the model the recipe asks for. Sets become $\mathcal{S} = \mathcal{A} \to \mathsf{Bool}$ for some $\mathcal{A} : \mathsf{Set}\ \ell$, the operations become opaque functions

```
opaque
  ∅ : 𝒮
  ∅ a = false
  _∪_ : 𝒮 → 𝒮 → 𝒮
  (A ∪ B) a = A a ∨ B a
```

and laws such as $(A \cup \emptyset) \equiv A$ become provable and registrable. The boolean ring interpretation [21] is a natural starting point for the rule set, but it needs rewriting modulo associativity and commutativity. Dedukti [8] supports this and Agda does not, which is the missing ingredient.

Implications for Intrinsically Typed Syntax. Complex intrinsically typed syntax, where substitution already appears in the type index, complicates proofs significantly. Without rewriting, substitutions enjoy few definitional equalities, so every manipulation of a proof over an expression whose type involves a substitution has to go through the transfer lemma (subst in Agda). This is the phenomenon known as transport hell, and it is a long-standing issue [9]. Intrinsically typed System F has been formalized in Agda before, executably and in full [11], and a binary logical relation over it [30] is a close example of the cost. Type-in-type and expression-in-expression reasoning stays easy, but type-in-expression substitution also affects the index of the expression substituted into, and the authors spend thousands of lines on the resulting transports. Rewriting the σ-calculus rules on the syntax of the typing index should remove them, and should generalize to other intrinsically typed formalizations.

5 Related Work

POPLmark solutions. Fifteen solutions to the Challenge have been published. Of those, four ever attempted the records extension, three the pattern-matching extension, and four Part 3. Berghofer's Isabelle/HOL de Bruijn development is the only one covering Parts 1, 2 and 3 together, and it animates the congruence-rule presentation because its code generator cannot execute the challenge's evaluation contexts. We cover 1A, 1B, 2A and 2B, and answer Part 3 as a decision procedure for the evaluation-context relation, machine-checking all seven tests of the challenge's own `step.poplmark`, at the cost of running on 2B without the pattern `let`. On POPLmark Reloaded we join Allais's generic-syntax Agda solution, Beluga and the two Rocq Autosubst solutions as a complete STLC and STLC+ solution. Allais additionally proves completeness of the inductive characterization of strong normalization, which we do not. The nearest neighbours in technique are Schäfer and Stark's Rocq de Bruijn solutions, by the authors of Autosubst and of the Reloaded benchmark. Coverage is not the contribution. It is what makes the claim about the substitution reasoning underneath it testable.

We have discussed Autosubst and related research [32, 35, 36], user-defined rewrite rules [13, 15, 24] and opaque definitions [18] throughout this work.

In recent work on local rewrite rules [25], the authors also rewrite σ-calculus rules with support for first-class renamings in Rocq. They too model renamings and substitutions as functions, but cannot rewrite all laws against that model, because with local rewrite rules the instantiation of the equations must hold definitionally rather than propositionally to maintain consistency. Rocq also has global rewrite rules [24], and we expect our approach to translate. Rocq, however, cannot instantiate rewrite rules with proofs, so the consistency guarantee would be lost.

Definitional substitution without rewriting. Kaposi and Pujet [22] meet the same tension from the semantic side. They present the initial category with families so that substitutions are metatheoretical functions, which makes the equations of the substitution calculus hold by computation and removes the transports that an indexed-inductive presentation forces. The mechanism is the one we exploit, namely that a function model computes, but it is deployed without rewrite rules, so which equations become definitional is fixed by the encoding rather than chosen. The η-laws are not among them, which is the boundary we reach from the syntactic side. Chen, Nordvall Forsberg and Tsai [12] give an intrinsic representation of type theory in Cubical Agda as quotient inductive-inductive-recursive types, and obtain a syntax that needs no transports, no postulates and no custom rewrite rules. Where they replace the rewrite rules by a quotient, we keep the plain inductive type and pay for it with a confluence obligation.

Generators for binder boilerplate predate and accompany Autosubst. Needle & Knot [23] generates the boilerplate for complex binding forms, including the relational lemmas that Autosubst leaves out, but does not aim at an equational theory of substitution and so has nothing to orient. Strongly typed term representations in Rocq [10] are the ancestor of the scoped syntax we use.

Another line of research concerns generic programming approaches [4, 5] that build syntax from general building blocks supporting sums, products and binders, and thereby abstract over renamings and substitutions. The approach carries over to a scoped, multi-sorted setting [29]. We believe generic programming and rewriting the σ-calculus are orthogonal, and that our rule set could be applied to the universe of syntaxes with binding, which would make external code generation unnecessary.

The Abstract Binding Trees library for Agda shares a goal with Autosubst and is used, for instance, in Siek's work [34]. It builds an object language from a generic syntax, but supports only extrinsic scoping and single-sorted syntax, and focuses on generic theorems about substitution and on abstractions over predicates. It also offers incomplete experimental support for rewriting with σ-calculus laws, but misses rules and therefore does not pass the confluence check.

Wadler experiments with explicit substitutions and implements weakening as a constructor, thereby avoiding renamings [39]. He rewrites some of the σ-calculus laws but does not pass the confluence checker. Section 1 discussed what that representation costs. He also represents substitutions as vectors, and so cannot instantiate the full rule set with proofs.

Wadler and colleagues also investigate a uniform treatment of renamings and substitutions in a simply typed setting [6], originating in work by McBride [27]. It unifies repeated similar lemmas, merging all four compositionality lemmas into one, by hiding the structural dependence of renamings and substitutions in a uniform, Mode-indexed vector, and lets the termination checker see that dependence where it matters. Saffrich [29] instead hides it behind instance resolution, in a scoped setting where the uniform structure is again a function, and adds reflection-based code generation for the syntax-dependent lemmas, which could be adopted here in place of our external generator.

Adapting our approach to a setting where renamings and substitutions are treated uniformly is a logical next step, but the equational theory would have to be extended in non-obvious ways to support uniform proofs over both. It would also require a two-level rewrite system, one on the type level, normalizing expressions in the lattice of Modes generated by $\mathsf{V} \sqsubseteq \mathsf{T}$ [6], and one for the adapted σ-calculus. In our experiments the interaction of the two layers made confluence hard to preserve.

6 Conclusion

We have shown how to turn the normalization procedure of the σ-calculus into definitional equalities in Agda. The central tension is that two contrary things are wanted at once: full computational behavior, so that the equational theory can be instantiated with proofs, and symbolic behavior, so that the rules are eligible for rewriting and higher-order pattern unification. Opaque definitions resolve it.

The theory we register is $\sigma_{\Uparrow}$ instantiated twice, once for renamings and once for substitutions, together with the rules that first-class renamings force between the two. Which laws survive is not a matter of preference. Native inductive variables and surjective pairing cannot coexist in one rewrite system, and that single fact fixes most of the design. The resulting system is locally confluent in Agda and confluent and terminating by external proof, a generator emits it from a syntax description, and the POPLmark Challenge and POPLmark Reloaded exercise it. One law resists our orientation, and we have said precisely why.

References

[1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. 1989. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) *(POPL '90)*. Association for Computing Machinery, New York, NY, USA, 31–46. doi:10.1145/96709.96712

[2] Andreas Abel and Brigitte Pientka. 2011. Higher-Order Dynamic Pattern Unification for Dependent Types and Records. In *Typed Lambda Calculi and Applications*, Luke Ong (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 10--26.

[3] Robin Adams. 2004. Formalized metatheory with terms represented by an indexed family of types. In *Proceedings of the 2004 International Conference on Types for Proofs and Programs* (Jouy-en-Josas, France) *(TYPES'04)*. Springer-Verlag, Berlin, Heidelberg, 1–16. doi:10.1007/11617990_1

[4] Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. 2021. A Type and Scope Safe Universe of Syntaxes with Binding: Their Semantics and Proofs. arXiv:2001.11001 [cs.PL] https://arxiv.org/abs/2001.11001

[5] Guillaume Allais, James Chapman, Conor McBride, and James McKinna. 2017. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs* (Paris, France) *(CPP 2017)*. Association for Computing Machinery, New York, NY, USA, 195–207. doi:10.1145/3018610.3018613

[6] Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. 2025. Substitution without copy and paste. https://nottingham-repository.worktribe.com/output/51884690/substitution-without-copy-and-paste. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)* (2025-07-19). Birmingham, UK.

[7] Thorsten Altenkirch and Ambrus Kaposi. 2016. Type theory in type theory using quotient inductive types. *SIGPLAN Not.* 51, 1 (Jan. 2016), 18–29. doi:10.1145/2914770.2837638

[8] Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. 2023. Dedukti: a Logical Framework based on the $\lambda\Pi$-Calculus Modulo Theory. arXiv:2311.07185 [cs.LO] https://arxiv.org/abs/2311.07185

[9] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *Journal of Automated Reasoning - JAR* 49 (08 2012), 1--19. doi:10.1007/s10817-011-9219-0

[10] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *Journal of Automated Reasoning - JAR* 49 (08 2012), 1--19. doi:10.1007/s10817-011-9219-0

[11] James Chapman, Roman Kireev, Chad Nester, and Philip Wadler. 2019. System F in Agda, for Fun and Profit. In *Mathematics of Program Construction: 13th International Conference, MPC 2019, Porto, Portugal, October 7–9, 2019, Proceedings* (Porto, Portugal). Springer-Verlag, Berlin, Heidelberg, 255–297. doi:10.1007/978-3-030-33636-3_10

[12] Liang-Ting Chen, Fredrik Nordvall Forsberg, and Tzu-Chun Tsai. 2026. Can We Formalise Type Theory Intrinsically without Any Compromise? A Case Study in Cubical Agda. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Rennes, France) *(CPP '26)*. Association for Computing Machinery, New York, NY, USA, 201–215. doi:10.1145/3779031.3779090

[13] Jesper Cockx. 2020. Type Theory Unchained: Extending Agda with User-Defined Rewrite Rules. In *25th International Conference on Types for Proofs and Programs (TYPES 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 175)*, Marc Bezem and Assia Mahboubi (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1--2:27. doi:10.4230/LIPIcs.TYPES.2019.2

[14] Jesper Cockx and Andreas Abel. 2018. Elaborating dependent (co)pattern matching. *Proc. ACM Program. Lang.* 2, ICFP, Article 75 (July 2018), 30 pages. doi:10.1145/3236770

[15] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. 2021. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.* 5, POPL, Article 60 (Jan. 2021), 29 pages. doi:10.1145/3434341

[16] Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. 1996. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM* 43, 2 (March 1996), 362–397. doi:10.1145/226643.226675

[17] Jean-Yves Girard. 1972. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur.* PhD thesis. Éditeur inconnu. https://www.cs.cmu.edu/~kw/scans/girard72thesis.pdf

[18] Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. 2022. Controlling unfolding in type theory. *arXiv preprint arXiv:2210.05420* (2022). https://arxiv.org/abs/2210.05420

[19] Thérèse Hardin, Luc Maranget, and Bruno Pagano. 1998. Functional runtime systems within the lambda-sigma calculus. *Journal of Functional Programming* 8, 2 (1998), 131--176. doi:10.1017/S0956796898003001

[20] Martin Hofmann. 1997. Syntax and semantics of dependent types. In *Extensional Constructs in Intensional Type Theory.* Springer, 13--54.

[21] Jieh Hsiang. 1985. Refutational theorem proving using term-rewriting systems. *Artificial Intelligence* 25, 3 (1985), 255--300. doi:10.1016/0004-3702(85)90074-8

[22] Ambrus Kaposi and Loïc Pujet. 2025. Type Theory in Type Theory using a Strictified Syntax. *Proc. ACM Program. Lang.* 9, ICFP, Article 266 (Aug. 2025), 31 pages. doi:10.1145/3747535

[23] Steven Keuchel, Stephanie Weirich, and Tom Schrijvers. 2016. Needle & Knot: Binder Boilerplate Tied Up. In *Proceedings of the 25th European Symposium on Programming Languages and Systems - Volume 9632.* Springer-Verlag, Berlin, Heidelberg, 419–445. doi:10.1007/978-3-662-49498-1_17

[24] Yann Leray, Gaëtan Gilbert, Nicolas Tabareau, and Théo Winterhalter. 2024. The Rewster: Type Preserving Rewrite Rules for the Coq Proof Assistant. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1--26:18. doi:10.4230/LIPIcs.ITP.2024.26

[25] Yann Leray and Théo Winterhalter. 2026. Encode the Cake and Eat It Too: Controlling Computation in Type Theory, Locally. *Proc. ACM Program. Lang.* 10, POPL, Article 62 (Jan. 2026), 30 pages. doi:10.1145/3776704

[26] Per Martin-Löf. 1975. An Intuitionistic Theory of Types: Predicative Part. In *Logic Colloquium '73*, H.E. Rose and J.C. Shepherdson (Eds.). Studies in Logic and the Foundations of Mathematics, Vol. 80. Elsevier, 73--118. doi:10.1016/S0049-237X(08)71945-1

[27] Conor McBride. 2005. Type-preserving renaming and substitution. (2005). http://strictlypositive.org/ren-sub.pdf Unpublished manuscript.

[28] John C. Reynolds. 1974. Towards a theory of type structure. In *Programming Symposium*, B. Robinet (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 408--425. https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/theotypestr.pdf

[29] Hannes Saffrich. 2024. Abstractions for Multi-Sorted Substitutions. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 32:1--32:19. doi:10.4230/LIPIcs.ITP.2024.32

[30] Hannes Saffrich, Peter Thiemann, and Marius Weidner. 2024. Intrinsically Typed Syntax, a Logical Relation, and the Scourge of the Transfer Lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development* (Milan, Italy) *(TyDe 2024)*. Association for Computing Machinery, New York, NY, USA, 2–15. doi:10.1145/3678000.3678201

[31] Steven Schäfer, Gert Smolka, and Tobias Tebbi. 2015. Completeness and Decidability of de Bruijn Substitution Algebra in Coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs* (Mumbai, India) *(CPP '15)*. Association for Computing Machinery, New York, NY, USA, 67–73. doi:10.1145/2676724.2693163

[32] Steven Schäfer, Tobias Tebbi, and Gert Smolka. 2015. Autosubst: Reasoning with de Bruijn Terms and Parallel Substitutions. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 359--374. https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf

[33] Kiraku Shintani and Nao Hirokawa. 2024. Compositional Confluence Criteria. *Logical Methods in Computer Science* Volume 20, Issue 1 (2024). doi:10.46298/lmcs-20(1:6)2024

[34] Jeremy G. Siek and Tianyu Chen. 2021. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming* 31 (2021), e30. doi:10.1017/S0956796821000241

[35] Kathrin Stark. 2020. *Mechanising Syntax with Binders in Coq.* Ph. D. Dissertation. Saarland University, Saarbrücken, Germany. https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.).

[36] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: reasoning with multi-sorted de Bruijn terms and vector substitutions *(CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 166–180. doi:10.1145/3293880.3294101

[37] Christian Urban, Stefan Berghofer, and Cezary Kaliszyk. 2013. Nominal 2. *Archive of Formal Proofs* (February 2013). https://isa-afp.org/entries/Nominal2.html, Formal proof development.

[38] Christian Urban and Christine Tasson. 2005. Nominal techniques in Isabelle/HOL. In *Proceedings of the 20th International Conference on Automated Deduction* (Tallinn, Estonia) *(CADE' 20)*. Springer-Verlag, Berlin, Heidelberg, 38–53. doi:10.1007/11532231_4

[39] Philip Wadler. 2024. Explicit Weakening. *Electronic Proceedings in Theoretical Computer Science* 413 (Nov. 2024), 15–26. doi:10.4204/eptcs.413.2